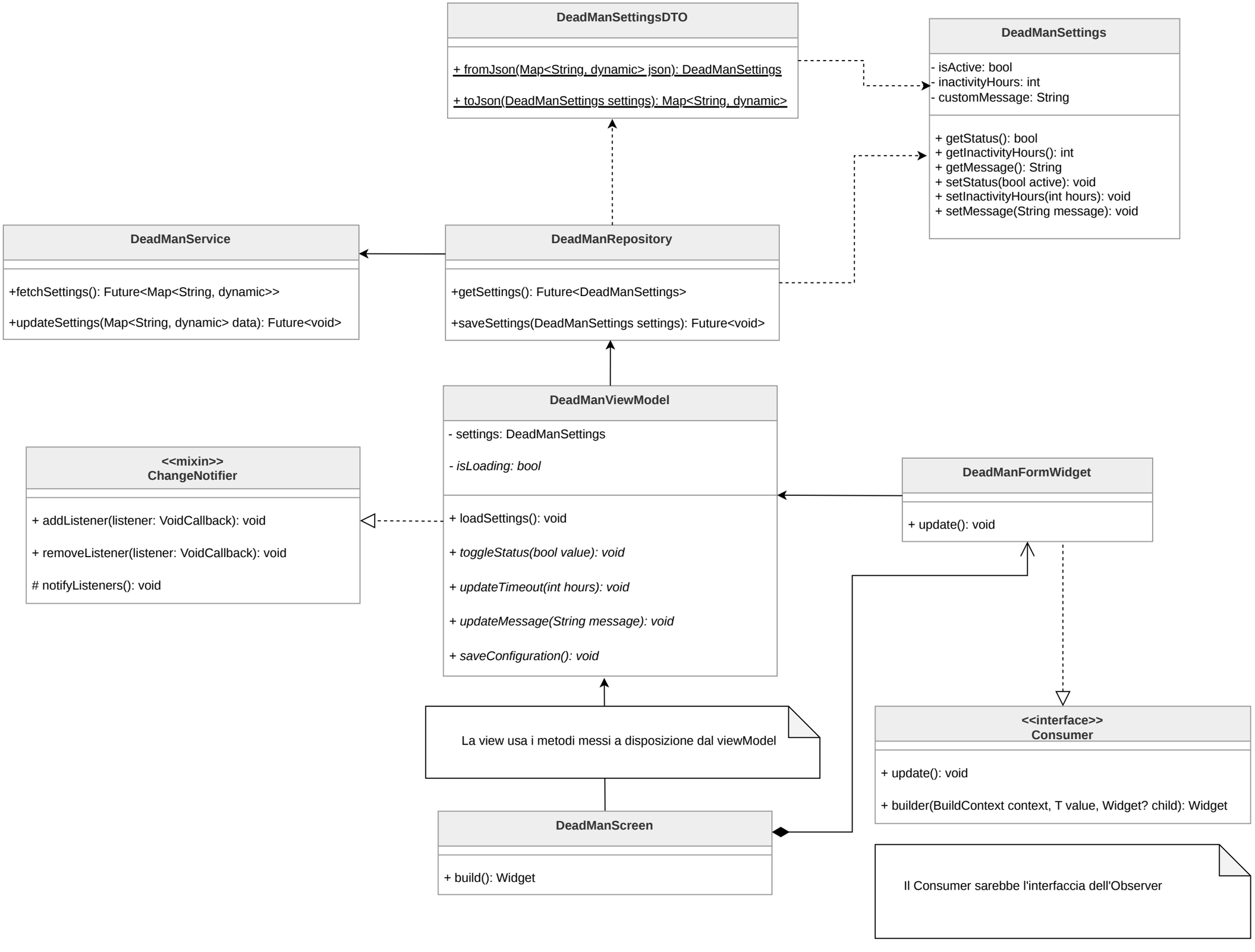
DeadManSettingsDTO
+ fromJson(Map<String, dynamic> json): DeadManSettings
+ toJson(DeadManSettings settings): Map<String, dynamic>
DeadManSettings
- isActive: bool
- inactivityHours: int
- customMessage: String
+ getStatus(): bool
+ getInactivityHours(): int
+ getMessage(): String
+ setStatus(bool active): void
+ setInactivityHours(int hours): void
+ setMessage(String message): void
DeadManService
+fetchSettings(): Future<Map<String, dynamic>>
+updateSettings(Map<String, dynamic> data): Future<void>
DeadManRepository
+getSettings(): Future<DeadManSettings>
+saveSettings(DeadManSettings settings): Future<void>
DeadManViewModel
- settings: DeadManSettings
- isLoading: bool
+ loadSettings(): void
+ toggleStatus(bool value): void
+ updateTimeout(int hours): void
+ updateMessage(String message): void
+ saveConfiguration(): void
<<mixin>>
ChangeNotifier
+ addListener(listener: VoidCallback): void
+ removeListener(listener: VoidCallback): void
# notifyListeners(): void
DeadManFormWidget
+ update(): void
La view usa i metodi messi a disposizione dal viewModel
DeadManScreen
+ build(): Widget
<<interface>>
Consumer
+ update(): void
+ builder(BuildContext context, T value, Widget? child): Widget
Il Consumer sarebbe l'interfaccia dell'Observer